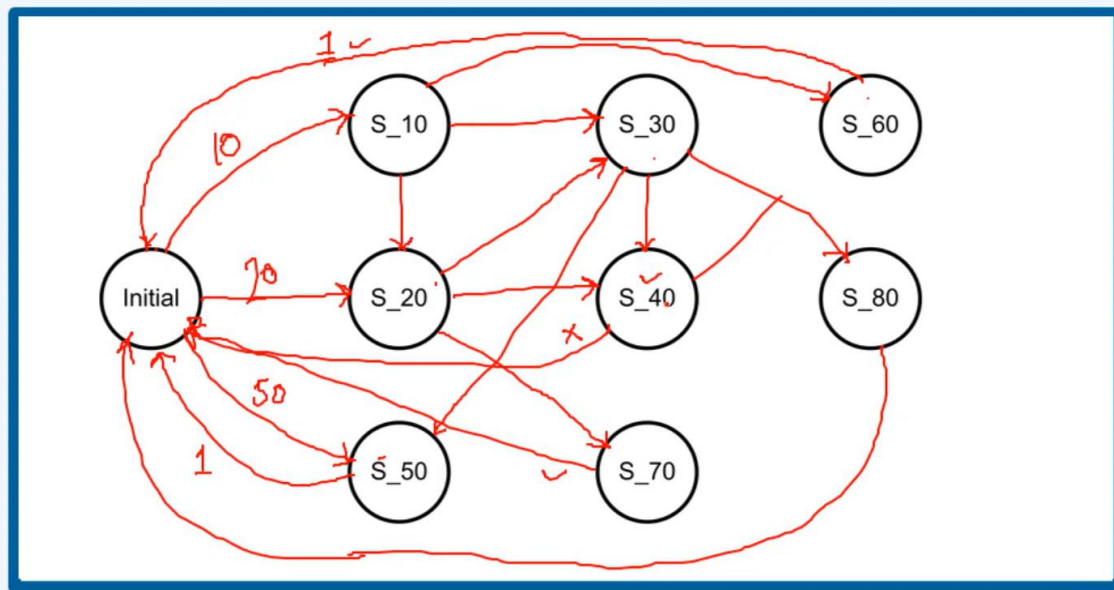


Project – Vending Machine Using FSM

Problem statement

- A vending machine automates the product sale at public places.
- Multiple products can be priced at different prices.
- The user enters money and select the product.
- The machine dispenses the product and gives the change if applicable.



Finite state machine for a Vending machine

Verilog design

- The Vending machine will be designed using the concept of Finite State Machines (FSM).
- The inputs are fixed as 3 different currency notes, so 2-bits are allocated for the input.
- The number of states and transitions are fixed before the coding.
- With every positive edge of the clock the machine reads the input and decides the next state.
- If any state is having a sum of more than 40 rupees, the product is dispensed.

Verilog Code:

```
module Vending_machine(coin, clk, reset, Z, change_given);

input wire clk, reset;

input wire [1:0] coin;

output reg Z, change_given;

parameter Sin=4'b0000, S10 = 4'b0001, S20 = 4'b0010, S30 = 4'b0011,
S40 = 4'b0100, S50 = 4'b0101, S60 = 4'b0110, S70 = 4'b0111, S80 = 4'b1000;
parameter ten = 2'b00, twenty = 2'b01, fifty = 2'b10;

reg [3:0] state, next_state;

always @(posedge clk, posedge reset)

begin

if(reset==1)

state <= Sin;

else

state <= next_state;

end

always @(state, coin)

begin

case(state)

Sin:

begin

if(coin==ten)

next_state = S10;

else if (coin==twenty)

next_state = S20;

else
```

```
    next_state = S50;
```

```
end
```

```
S10:
```

```
begin
```

```
    if(coin==ten)
```

```
        next_state = S20;
```

```
    else if (coin==twenty)
```

```
        next_state = S30;
```

```
    else
```

```
        next_state = S60;
```

```
end
```

```
S20:
```

```
begin
```

```
    if(coin==ten)
```

```
        next_state = S30;
```

```
    else if (coin==twenty)
```

```
        next_state = S40;
```

```
    else
```

```
        next_state = S70;
```

```
end
```

```
S30:
```

```
begin
```

```
    if(coin==ten)
```

```
        next_state = S40;
```

```
    else if (coin==twenty)
```

```
    next_state = S50;
else
    next_state = S80;
end
```

```
S40: next_state = Sin;
S50: next_state = Sin;
S60: next_state = Sin;
S70: next_state = Sin;
S80: next_state = Sin;
default: next_state = Sin;
endcase
```

```
end
```

```
// Now the case for allocating the output values
```

```
always @(state,coin)
```

```
begin
```

```
case(state)
```

```
Sin: begin Z = 0; change_given = 0; end
```

```
S10: begin Z = 0; change_given = 0; end
```

```
S20: begin Z = 0; change_given = 0; end
```

```
S30: begin Z = 0; change_given = 0; end
```

```
S40: begin Z = 1; change_given = 0; end
```

```
S50: begin Z = 1; change_given = 1; end
```

```
S60: begin Z = 1; change_given = 1; end
```

```
S70: begin Z = 1; change_given = 1; end
```

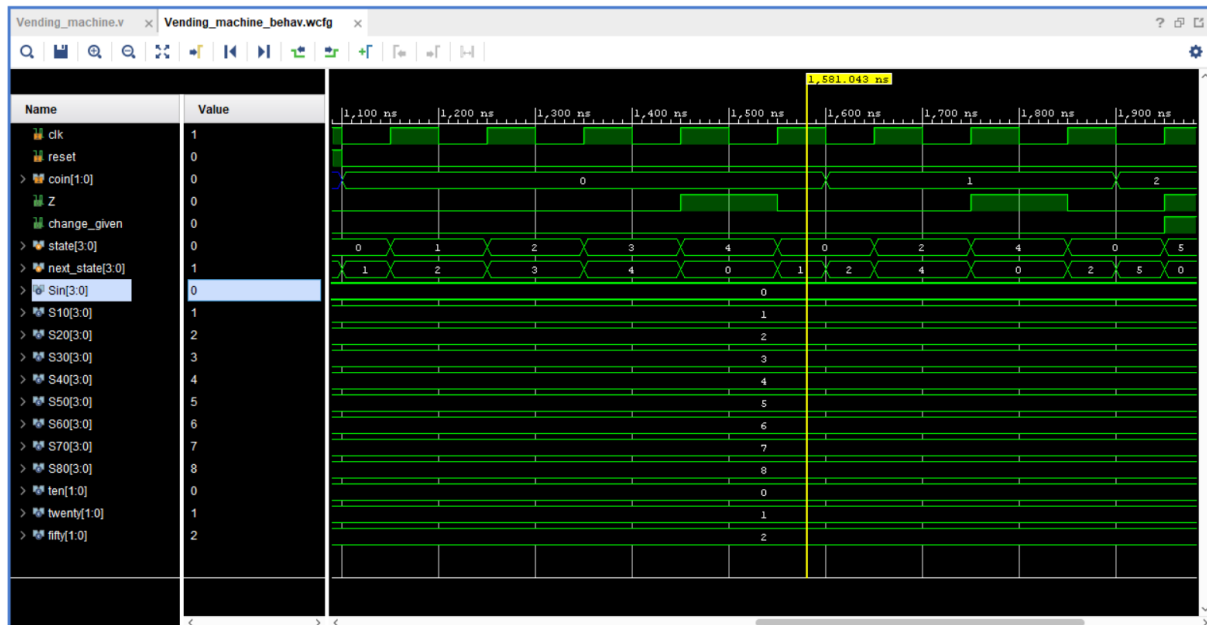
```
S80: begin Z = 1; change_given = 1; end
```

```
endcase
```

end

endmodule

Simulation Waveform :



Tools used and Language Used:

Xilinx Vivado and Verilog.